

Klavis: Keystroke Biometric Hardware Token

ECE 547/647 Security Engineering
University of Massachusetts Amherst
May 2026

Team 2

Santiago Alzate, Kyle Belanger, Tomer Carmeli, Jeffrey Kidwell

Abstract

Hardware authentication tokens such as FIDO2-compliant USB security keys provide strong cryptographic guarantees, but operate under the critical assumption that possession of the token equals proof of identity. This leaves systems vulnerable to scenarios where a token is stolen, borrowed, or otherwise used by an unauthorized person. Existing keystroke biometric systems, while promising as a behavioral “something you are” factor, are designed for large static datasets and are ill-suited for real world deployment where only a handful of enrollment samples are practical.

KLAVIS is a keystroke biometric hardware token that addresses this gap by integrating adaptive behavioral authentication directly into the FIDO2 authentication flow. The system captures keystroke timing features such as dwell time, flight time, and digraph latency as the user types a fixed catchphrase, such as their chosen password. KLAVIS then uses these features to verify that the person holding the token is its legitimate owner before allowing the WebAuthn challenge response sequence to proceed.

The core authentication engine uses a Scaled Manhattan Distance algorithm with per-feature normalization, paired with a progressive enrollment and template adaptation system that allows user profiles to mature through continued use. Getting this to work effectively at small data scales, where traditional machine learning approaches fail and profile poisoning becomes a threat, is the critical challenge that motivated our design decisions for this project.

KLAVIS demonstrates that strong, adaptive behavioral biometrics can be practically integrated into commodity hardware security tokens without requiring large training datasets or complex infrastructure, offering a deployable path toward truly possession and identity bound authentication for everyday users.

1. Introduction

Klavis is a keystroke biometric authentication software. It was made in the hopes of providing more security to an already existing web authentication service. Making it open-source and available to users through simple executables and downloads was part of our mission to also provide a good user experience; the guis used also contribute to the user friendly experience. Keystroke biometrics have been researched and studied over many decades. Many algorithms exist to time such biometrics and analyse them. We aimed to make security more about something you are rather than something you have because something you have may be discovered or compromised and cause critical security issues. Something you are is not as easily compromised and provides protection against the aforementioned situation.

Our approach to Klavis is simple. Research algorithms used for keystroke biometrics, find which one works the best for our implementation and use it. Once we have the algorithm ready to use, we make any improvements/adjustments needed along the way. For example, our adaptive enrollment that weighs the users login attempt to previous login attempts and checks whether it is within an acceptable range that dynamically changes according to the number of samples available. From there, we bought a SoloKeys USB off of their website as well as made GUIs for both the developers (ourselves) and users. We expect to have a GitHub page that has download links for both Windows and Linux for the application versions of Klavis. This way users who aren't interested in the actual code can simply just download and use the software rather than learning to use a terminal to run a python script. The application should be visually appealing and provide the user with an enrollment period if they've never used Klavis before. If they have used it before, they should be able to login to their profile. The authentication used in Klavis should have a small false acceptance rate and false rejection rate as seen in our results section. Along with this, we plan to integrate Klavis into the firmware of the SoloKeys USB, so that it always authenticates when the USB is plugged in.

This project demonstrates the fundamentals of security taught throughout the course in various ways. It shows the importance of 2FA, adding a layer of security that does not solely rely on a single password that could be compromised. Klavis also exists on the premise of one of the fallacies that "Proprietary security algorithms are more secure since nobody knows how they work". We made it open source so that anyone can access the algorithm we used and test it out for themselves and try to compromise it so that it can be made more secure. The web authentication used in the SoloKeys USB also uses standard encryption algorithms like hashing algorithms and web authentication algorithms that make it FIDO2 compliant.

The rest of this report is organized by the following sections (in order): Related works, Methodology and Tools, Partitioning of work and schedule, Experiments and Results, Demo video link, Analysis and Reflection, and Conclusion and Future works.

2. Related Work

The field of biometric authentication encompasses a broad range of approaches, from hardware intensive physiological systems to behavioral methods that leverage the natural patterns in how people interact with devices. This section reviews the foundational work that

motivated KLAVIS, covering prior keystroke biometric hardware systems, the benchmark dataset and methodology that helped build our algorithm design, and additional algorithmic approaches that helped shape our implementation decisions.

2.1 Keystroke Biometrics on a Hardware Token - Cockell and Halak (2019)

One of the most directly relevant prior works to KLAVIS is the hardware token prototype proposed by Cockell and Halak, which investigates the feasibility of integrating keystroke dynamics into a self contained, portable authentication device [1]. The authors identify a fundamental weakness in conventional authentication systems, that password based mechanisms verify only whether the correct credential is presented, not whether the legitimate user is the one presenting it. Hardware tokens address possession but not identity. More advanced biometric systems like fingerprint or retina scanners are effective, but introduce increases in hardware complexity and cost. Their proposed system fits in a practical middle ground by extracting behavioral biometric signatures passively from the act of entering a password.

Their prototype is made up of a ten button keypad instrumented with linear motion potentiometers that capture the pressure and displacement profile of each button press, feeding analog signals into an ADC and then into a Raspberry Pi for processing. Rather than relying solely on digital timing data, the system extracts six characteristics per keypress: maximum displacement, press length, plateau length, attack value, decay value, and mean average displacement. This feature set enables more complex user profiling than timing only approaches. Two verification algorithms were evaluated, a simple percentage difference method and a probability estimation method, with the latter working more effectively, by modeling the distribution of both training and background data to compute per-press user probability scores.

Evaluation was conducted across 32 participants with over 500 password repetitions, using a 10 digit numeric password. Ten users provided training data and were tested against both correct and imposter attempts. Results showed that the probability based algorithm achieved an average error rate of 4.5%, with six of ten users achieving zero false accepts and zero false rejects [1]. Important to note, no single fixed threshold was effective across all users, suggesting the need for user-specific calibration. The authors conclude that their results could be improved by implementing machine learning approaches.

This paper helped us put our project into perspective while giving us specific pitfalls to address in our own implementation. The core motivation, that a possession of a token is insufficient to verify identity, goes hand-in-hand with our problem statement that we aim to address. However, whereas Cockell and Halak focus on a custom built keypad with pressure sensing, KLAVIS operates on a standard full keyboard with a fixed catchphrase, making it applicable to any existing computing environment without specialized hardware. Their observation expressing the need for user-specific calibration for thresholds directly motivates our implementation of an adaptive enrollment method. This is necessary to support both consistent and inconsistent typists by properly growing their profiles based on the strength of the profile at each step.

2.2 Keystroke Dynamics: Concepts, Techniques, and Applications - Shadman (2025)

This paper focuses on presenting information about keystroke analysis. It contains lists of not only relevant research papers, but datasets and algorithms that pertain to making a project. Along with this, it gives many suggestions on how to implement a keystroke algorithm using ML, statistical analysis, and expression tracking as well as their advantages and disadvantages.

One of the main points throughout this article is the difference between the different types of typing prompts. There is Free text, Fixed text, and Semi-fixed text. Free text is basically the user can type whatever they want. Fixed text is when the user is given a specific word or jumble of letters and characters to type and has to strictly follow the given phrase. Semi fixed text is when the user has a strict phrase, but they have many phrases they can choose from. An example of this type of text given in the article is terminal commands. They are set in how you use them, but they can have many variations to choose from. For our purposes, we chose to use Fixed text for Klavis, since the overhead of the other two would be very complicated to implement in an authentication algorithm. This is especially true when we think about how to make that work with a plug in USB being detected.

Another interesting part of this article is the use of terminology. For non-continuous authentication, we use the terms false acceptance rate (FAR) and false rejection rate (FRR) to provide statistics for how many users are let in that aren't supposed to and for how many users get rejected even when it is their account. The interesting part is that continuous time authentication uses different terminology: average number of imposter actions (ANIA) and average number of genuine actions (ANGA). They use this terminology because if you were to automatically assume something is malicious off of one singular value/moment, you wouldn't have probable cause to do so because continuous time has inconsistencies like when the user is typing or when they are away from their keyboard.

2.3 Keystroke Dynamics for User Authentication - Sharma, Jureček, Stamp (2023)

This paper builds on the underlying principles of keystroke biometrics from the papers by Shadman (2.2) and Cockell, Halak (2.1) and analyzes various statistical models to identify users based on their typing behavior as an alternate authentication method to standard passwords. Traditional authentication systems, such as passwords are able to authenticate credentials, but not the user themselves. This means that if a password gets leaked, an attacker can easily gain access to a system. Keystroke authentication acts differently, as each user has a unique typing pattern, making it much more difficult for an impostor to gain access to a system. In this paper, the authors propose three different models to identify users based on their keystrokes as well as analyze the overall accuracy of each method.

The researchers' tests first started off by analyzing the widely popular Buffalo free-text dataset, which contains over 17,000 keystrokes per user for 148 users. There are six different keystroke patterns that were looked at: total time between first key press and next key release,

key press time, key-up key-down time, key-down key-down time, key-up key-down, and key-up key-up time. To support this much data, an image-like data structure called a Keystroke Dynamic Image (KDI) was used to store the parameters as a multi-dimensional matrix. The way the KDI works is that each pattern, such as key-up key-down, gets its own matrix. The rows in the matrix represent the first character inputted in a sequence by the user while the column represents the next character. Through this representation, the data can efficiently be passed into two algorithms: Convolutional Neural Networks (CNN) and Random Forest Trees. With the CNN, it was trained on the image-like keystroke features directly, where the goal was a multiclass classification of all users. On the other hand, Random Forest used a modified version of the KDI.

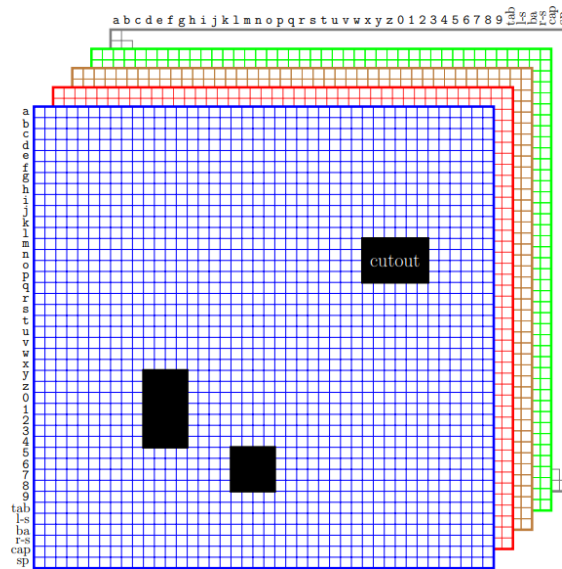


Figure 1: Image-like data structure for keystroke data

To evaluate the algorithms, the authors looked at how accurate each method could identify a user based solely on the given information from the dataset. The CNN ended up with an accuracy of 78%, while the Random Forest method achieved 93%. These results were notable since no external hardware was required to gather this information, relying only on basic keyboard typing information, proving that keystroke dynamic identification is possible through a sole software interface. This proved to be really helpful for our project, as we now had a reference for designing efficient and accurate algorithms to detect impostor intrusion attempts.

3. Methodology and Tools

3.1 System Architecture Overview

KLAVIS is designed as a layered authentication system for SOLO2 tokens. It consists of a host-side daemon, a graphical user interface, and a local keystroke biometric authentication engine. The main goal of the architecture is to add an identity verification mechanism before the USB token can be used. In a normal FIDO2-style authentication flow, possession of the USB token is treated as sufficient proof that the correct user is present. KLAVIS strengthens this assumption by requiring the user to pass a keystroke biometric check before the authentication process is allowed to continue.

At a high level, the system begins when the Solo2 USB token is inserted into the user's computer. A host daemon continuously monitors the system for the presence of the USB device. Once the token is detected, the daemon launches the KLAVIS user interface. The interface presented to the user depends on the selected operating mode. In normal use, the user-facing GUI is launched and provides a simplified login and enrollment experience. In developer mode, the developer GUI is launched and exposes detailed logs, score, thresholds, benchmark tools, and visualization options for testing and analysis.

The KLAVIS software stack is organized around several core modules. The GUI layer is responsible for user interaction, including profile creation, passphrase entry, enrollment progress, authentication attempts, password reset, retraining, and benchmark collection. While the user types the passphrase, the GUI captures key press and key release events. These events are passed to the keystroke capture and feature extraction pipeline, where raw timing data is converted into biometric features such as dwell time, flight time, and digraph-level timing relationships.

Once the timing features are extracted, they are passed to the authentication engine. The engine uses a Scaled Manhattan Distance Algorithm to compare the current typing attempt against the stored profile for that user. Each user profile contains a stable bigram ordering, a

mean timing vector, and a mean absolute deviation vector. The mean vector represents the user's typical typing rhythm, while the MAD vector normalizes each feature according to how naturally variable it is. This allows the system to avoid over-penalizing timing features that are inconsistent for a legitimate user while still giving weight to features that are normally stable.

The profile manager is responsible for storing and updating user profiles locally. Each profile is saved as a JSON file and contains the information needed to authenticate future attempts, including the user's typing template, enrollment metadata, authentication history, benchmark history, and profile lifecycle state. This local design avoids the need for external servers or cloud infrastructure, which keeps the system lightweight and self-contained. It also fits the project's goal of building a practical authentication layer that could eventually be integrated directly into secure hardware tokens.

A key part of the architecture is the adaptive profile lifecycle. During initial enrollment, the system does not require a fixed number of samples for every user. Instead, adaptive enrollment collects between a minimum and maximum number of samples and stops early if the profile becomes statistically stable. After enrollment, the profile enters a growth phase, where high-confidence successful attempts are used to strengthen the user template. Once the profile has matured, the system transitions into an adaptation phase, where exponential moving averages allow the profile to slowly track natural changes in the user's typing behaviour over time.

To protect the profile from corruption, KLAVIS separates authentication from learning through a dual-threshold design. The authentication threshold determines whether the user is granted access, while the stricter learning threshold determines whether the attempt is sufficient to update a stored profile. This prevents borderline accepted attempts from influencing the profile and reduces the risk of profile poisoning, where an impostor or unusual typing attempt gradually shifts the biometric template away from the legitimate user.

The architecture also includes separate research and evaluation components. The dataset manager records timing data in a structured format for later analysis, while the benchmark and aggregation modules compute performance metrics such as false acceptance rate, false rejection rate, and equal error rate. Visualization modules then generate plots showing score distributions, threshold behaviour, profile maturation, and feature-level contributions. These tools are not required for normal authentication, but they are important for evaluating the system and tuning its parameters.

Overall, the klavis architecture separates the system into clear functional layers: USB token detection, user interaction, keystroke capture, feature extraction, biometric scoring, profile persistence, adaptive profile management, and benchmark visualization. This modular design made it possible to develop and test each part of the system independently while still producing an integrated authentication workflow. Although the current implementation runs the biometric decision on the host machine rather than directly on the Solo2 firmware, the architecture

preserves the intended security model: the token should only be usable after the person holding it has proven their typing behavior matches the enrolled owner.

3.2 Solo2 USB Hardware Token

The hardware token being used is a Solo2 USB security key ordered from the SoloKeys website. This device is fully compliant with the security protocols U2F and FIDO-2, making the implementation as an authentication service for external apps extremely smooth and secure. Within the USB, there is a security key that can be accessed. This is what makes the device FIDO-2 compliant, which is then passed on to external applications such as Google WebAuth.

3.3 Host Daemon

For the Daemon firmware, numerous Python packages were utilized. The firmware is constantly being run in the background on a user's computer and scans for when the Solo2 USB key is plugged into a USB port. This is done using two main packages: Windows Management Instrumentation (WMI) as well as Pyudev. Both of these libraries are used to make the firmware compatible with different operating systems. WMI is well-suited for Windows applications, while Pyudev is more optimized for Unix-based systems, including MacOS and Linux.

The firmware continues to search for the device until it has been plugged in. Once this event occurs, a graphical user interface (GUI) appears on the screen. What appears on the GUI is based on when the last time the USB was last registered. The way the script checks this is by looking for a json file in the same directory. Within that file, there will be an entry for the last time the device was registered. If the registration date exceeds 3 months or a json file does not exist, then the user will have to re-enroll again as part of our certificate process. This is done to account for user typing pattern changes that happen over time.

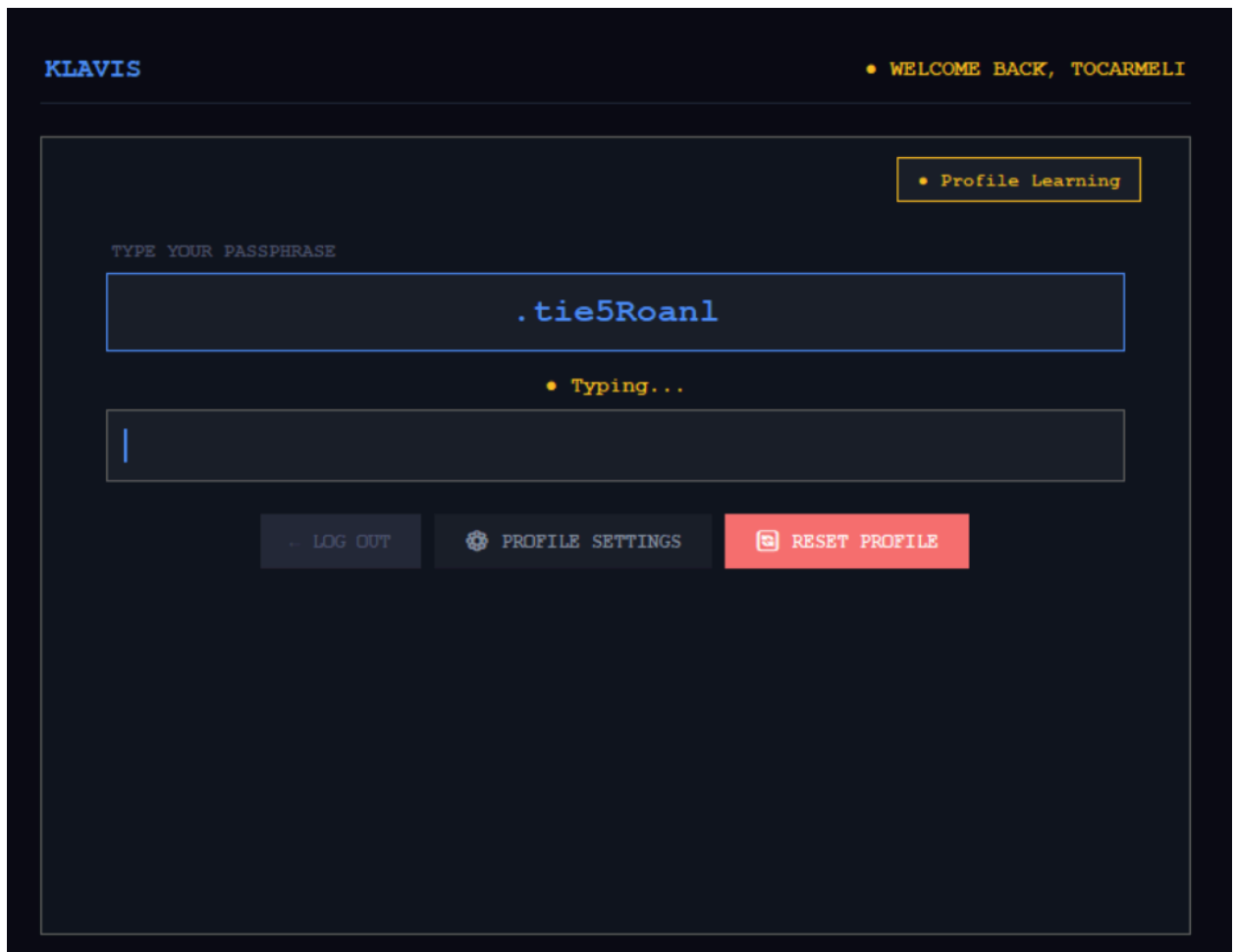


Figure 2: Screenshot of GUI user is prompted with Solo2 Key is plugged in and user logs in

3.4 Scaled Manhattan algorithm and MAD Normalization

KLAVIS uses a Scaled Manhattan Distance algorithm as its core biometric classifier. This algorithm was chosen because it is lightweight, interpretable, and effective with a small number of enrollment samples. Instead of requiring a large machine learning dataset, the system builds a user profile from repeated passphrase attempts by measuring timing features such as dwell time, or how long each key is held, and flight time, or the time between releasing one key and pressing the next. These features are organized into a fixed vector based on the bigrams in the user's passphrase.

During enrollment, KLAVIS computes an average timing vector for the user. This vector represents the user's typical typing rhythm. When the user later attempts to authenticate, their new timing vector is compared against this stored average. The basic comparison uses Manhattan distance, which adds up the absolute difference between each live timing feature and the corresponding stored feature. A smaller distance means the attempt is closer to the

enrolled user's typing pattern, while a larger distance suggests an impostor or abnormal attempt.

The algorithm is “scaled” using Mean Absolute Deviation, or MAD, normalization. MAD measures how much each feature naturally varies during enrollment. This matters because not every timing feature is equally consistent. Some key transitions may vary a lot even for the legitimate user, while others may be very stable. By dividing each feature difference by its MAD value, KLAVIS reduces the impact of naturally noisy features and increases the importance of features that are normally consistent.

The final output is a single score that determines whether the typing attempt should be accepted or rejected. Low scores indicate that the live attempt closely matches the enrolled profile, while high scores indicate that the timing behavior is different. This makes the algorithm a strong fit for KLAVIS because it can run locally, works with limited enrollment data, and produces scores that are easy to interpret during testing and debugging

3.5 Progressive and Adaptive Enrollment

Traditional keystroke biometric systems require a fixed number of enrollment samples before building a user profile. This number could be anywhere from hundreds to thousands of samples, which is impractical for a hardware token where a user would never tolerate typing a passphrase that number of times just to register. This would make our project as a whole unusable from a commercial sense, as users would give up before even enrolling. KLAVIS addresses this through a progressive and adaptive enrollment system that determines when enough samples have been collected based on statistical evidence of stability rather than a hardcoded count.

The enrollment process operates within bounded limits of a minimum of 8 samples and a maximum of 20 samples. Below the minimum, the system always continues collecting regardless of how strong the profile looks, since there is not yet enough data to make a meaningful stability judgment. Above the maximum, enrollment always terminates as a safety escape to prevent the system from asking indefinitely in edge cases for inconsistent typists. Between those bounds, every new sample triggers a stability evaluation that decides whether enrollment can stop early.

Stability is evaluated using two signals computed over a sliding window of the most recent 5 samples. The first is the coefficient of variation (CV), which measures how consistent the timing features are relative to their mean values. For each feature in the typing vector, CV is computed as the standard deviation divided by the absolute mean, and the median CV across all active features is used as the summary statistic. A lower median CV means the user's timing has converged. The second signal is the average self score, which is computed by building an engine from all collected samples and scoring the recent window against it using the same Manhattan scaled distance algorithm for authentication. A lower average self score means that the recent samples are consistent with the overall enrolled profile.

The CV calculation only considers features with non-zero mean values. Early versions of the system suffered from artificially reduced CV scores because bigrams that were zero padded

pulled the median down, making erratic typing appear falsely stable. A zero padded bigram was a result of features present in some attempts but not others due to typos or capture artifacts.

When both the median CV falls below 0.10 and the average self score falls below 0.80, enrollment is stopped early with a stable output. If the maximum sample count is reached without meeting those criteria, enrollment is stopped with the output `hit_maximum`. The stop reason and full stability metrics are persisted in the profile JSON under `enrollment_policy`, which keeps track of how each profile was built and supports future analysis of whether enrollment quality correlates with long term authentication performance. Consistent typists finish enrollment in 8 to 12 samples, while inconsistent typists may hit the maximum of 20. With these methods implemented, there is no need for manual tuning per user, and no risk of the system running forever.

3.6 Dual Threshold Design

The main vulnerability that adaptive biometric systems face is profile poisoning. This is the gradual corruption of a user's stored profile through accepting imposter attempts or borderline genuine attempts that do not accurately represent the user's typical typing. KLAIVIS operates with small enrollment samples and continues to update the profile during normal authentication use, which means this risk is amplified compared to static biometric systems that never modify their stored template. This is addressed in our project by using a dual threshold design that separates the decision of whether to grant access from the decision of whether to learn from an attempt. These two decisions use different thresholds and serve different purposes.

The first threshold is the authentication threshold, which determines whether an attempt is accepted or rejected. This threshold is set at 2.5 scaled Manhattan distance units during the growth phase, and loosens to 3.0 during the adaptation phase once the profile has matured. The tighter growth phase threshold reduces false accepts while the profile is still being established and has not yet seen enough data to reliably represent the user. Once the profile matures into the adaptation phase, the looser threshold of 3.0 reflects increased confidence in the stored model and reduces the chance of falsely rejecting the genuine user whose profile is now well characterized.

The second threshold is the learning threshold, set at 1.5 scaled Manhattan distance units, which is much stricter than the authentication threshold. Even if an attempt is accepted for access, it is only incorporated into the profile if its score falls below this learning threshold. This means borderline accepted attempts, with scores between 1.5 and the authentication threshold, grant access but do not influence the stored profile. Only confident, low scoring attempts that strongly resemble the enrolled profile are used to update the mean and MAD vectors.

The difference between these two thresholds creates a protected zone. An imposter who happens to type similarly enough to slip past the authentication threshold, which is rare but possible, would score somewhere in the range of 1.5 to 3.0, which grants access but does not poison the profile. This prevents an attacker from being able to pull a profile closer and closer to their typing pattern over time. A genuine user typing in an unusual way due to fatigue or distraction similarly gets access without this attempt pulling the profile away from their normal pattern. Only genuinely representative typing updates the stored model.

This design was motivated directly by a problem encountered during development. Early versions used a single threshold for both decisions, which led to us encountering the issue of profile poisoning over time. The dual threshold eliminated this degradation and stabilized profile maturation across extended use. The thresholds themselves were determined through benchmark testing across multiple user profiles and represent a starting point that can be tuned further as more benchmark data is collected across additional users.

3.7 GUI interface

KLAVIS implements two separate graphical user interfaces that share the same backend engine, profile management, and authentication logic while presenting the user with different levels of information and control to their respective audiences. Both GUIs use the same objects and read from and write to the same profile directory. A profile created or matured in one GUI is immediately accessible in the other. The USB daemon script `usb_find.py` accepts a command line argument to select which GUI to launch when the Solo2 token is detected, defaulting to the user GUI for normal deployment and switching to the developer GUI with the developer tag added to the terminal command for testing sessions.

The user facing interface (`user_gui.py`) was designed around the principle that the biometric system should be invisible to the end user. A person using KLAVIS should experience it as a simple passphrase-based login system, with the keystroke analysis happening in the background, not exposing any of the underlying scoring to the user.

The interface opens on a landing screen that presents two options, log in or create a profile, along with quick access buttons for any previously enrolled users. If the USB certificate is older than 90 days, a warning message appears on this screen prompting the user to retrain their profile. Different from the developer GUI's set passphrase, the user GUI asks the user to create a profile name and passphrase that meets defined complexity. The complexity we have chosen is a minimum of 8 characters, at least one uppercase letter, at least one digit, and at least one special character. These requirements mirror the complexity of the research passphrase ".tie5Roan!" used in the benchmark study we based the majority of this project on.

The enrollment and authentication screens present a much simpler display compared to the developer GUI, only showing simple dot based progress indicators and status indicators respectively. This leaves out the technical output like scores, thresholds, and error rates, which are stored in a log buffer that is only visible through a menu that is shown when "Ctrl+Shift+D" is pressed. This hidden menu is only for testing purposes, and would be removed if our product were to be shipped for consumer use.

The developer GUI displays a persistent live log panel on the right side of every screen that outputs every system event in real time. This makes it possible to observe exactly what the system is doing on every keystroke attempt without inspecting the profile JSON directly. The developer GUI also has a set catchphrase, as mentioned earlier, that is found to be statistically the best catchphrase for tracking user typing patterns. This allowed us to test our system with the most sound input, allowing us to tune the parameters before creating the user GUI.

The enrollment screen shows explicit sample counts against the maximum, and the log panel reports the stability reason for every sample including the median CV and average self score values that drive the adaptive stopping decision. The authentication screen displays the

current phase with full detail, including samples to go before transition and days elapsed since profile creation. This GUI also exposes benchmark functionality directly from the authentication screen rather than behind a hidden menu. This benchmark mode collects a configurable number of genuine and imposter attempts, displays FAR, FRR, and EER results, and generates a panel of 6 graphs displaying important testing information.

4. Partitioning of Work and Schedule

This section of our paper breaks down the individual contributions of each team member, and the schedule of when this work began and was completed. This consists of one subsection per group member, and provides a look into how we both worked independently and collectively for each task along the way.

4.1 Kyle Belanger

I began working on this project primarily doing research into what method we would choose to ultimately go with and change for our deliverables. During this research, I looked into a different approach than we ended up deciding on, that being using machine learning to train profiles. I looked into already existing libraries of collected user data using the same Manhattan algorithm to potentially train our own model on due to the limitation of not being able to collect a significant amount of data in our time constraint. This was ultimately replaced with our adaptive enrollment and dual threshold design.

Once we decided on this design method, I used what would in the end become the developer GUI that Santiago created to implement the adaptive enrollment system. This replaced the original fixed sample count enrollment, allowing the system to evaluate typing stability after each sample using the two signals talked about in the adaptive enrollment section. A key implementation fix that I faced was filtering out zero padded bigram features from the CV calculation, which was causing erratic typists to appear falsely stable due to consistently zero valued bigrams bringing the median CV score down. With this system, I also implemented the adaptive phase switching system that determines when a user profile graduates from the training to the growth phase. Rather than relying on a static sample count to determine when this switch would occur, I created new thresholds to actively cause this change to occur. This allows consistent typists to start growing their profile as soon as it's ready, rather than waiting for a specific count to be reached.

I also implemented the dual threshold system that separates the authentication decision from the profile learning decision. The authentication threshold determines whether access, while a separate stricter learning threshold is in place to determine if accepted attempts are actually used in the stored profile for learning. This prevents the profile from being poisoned, ensuring that borderline accepted attempts do not corrupt the stored mean and MAD vectors. The design was motivated directly by observed profile degradation in early testing when a single threshold was used for both decisions.

Lastly, I designed and built the user facing interface in `user_gui.py` as a clean replacement for the original developer GUI. The interface introduces a landing screen with login

and profile creation flows, a live password requirements checklist during profile creation, a simplified enrollment screen that hides all technical output from the end user, and a profile settings panel that allows passphrase changes and manual retraining. All scoring, threshold, comparisons, phase transitions, and stability metrics are written to a hidden internal log buffer that is hidden from the user. I also integrated the USB certificate system that Tomer created into the GUI, displaying a warning message when the user's profile must be retrained. I also updated Tomer's script to allow for both the user and developer GUI to be opened depending on a flag when running the `usb_find.py` script. I then passed this updated GUI and certificate implementation to Santiago for him to implement the arithmetic behind how certificate retraining is handled.

The timeline for the work I did for this project is divided into two halves. While I was waiting for the initial programs to be created by Santiago, I was doing the research into other methods. Once we settled on our current method, I then was able to take what was produced by him, and implement the new features described in this section during the second half of this project.

4.2 Jeffrey Kidwell

I began by reading the research papers that our group found. After we had all read the research papers and gotten a good idea of what we were aiming for with this project, I started to build a mock up of an algorithm I found on one of the papers. It was a Euclidian distance algorithm, rather than the Manhattan distance algorithm we used in our final version. This took about a day or two since I was unfamiliar with the algorithm, but it was mostly just averages and statistics, so it wasn't too hard to implement. After I had it working, I tested it out for a while. It worked pretty well, but it had a few problems. One problem was that the threshold was very hard to get right. Increasing or decreasing it significantly changed the results of the algorithm. It didn't need to be a big change either, very small changes in the threshold also affected it. One thing that was good about this algorithm though (compared to the Manhattan distance one) is that it didn't use as much data to perform the authentication. This meant that the JSON files produced for the profiles using this algorithm were slightly smaller than the ones from our current algorithm. This doesn't make that much of a difference, but for embedded systems it could be relevant?

The next thing I did was update the layout of the Github page. Tomer made the initial blank page and shared it with the group, so I basically decorated it and organized it. I made folders for the software, firmware, and any data we may get from testing. I added a description of the project, the fact that it's FIDO 2 compliant, and a brief mention of the SoloKeys company.

Once there was a working version of Klavis, I took some time to create a profile of my own so that other members in the group could use it for testing if needed. I ended up making a profile with 201 samples and posted it to the git page.

From there, my main job was to basically just read through and understand the code and make the user facing application of it. I ended up making two versions of the application and

posting them to github. The first version was the one we initially used to test out the algorithm and see what needs to be improved. The second version I posted was the finalized version updated with all the new features of our final implementation. I posted both a Windows and Linux version of the executable so that it's more accessible to users. The windows version of the application was pretty easy to make since my computer runs windows, but the Linux version was a bit of a hassle to make. It required using WSL, serial copying files into the correct path, making a virtual environment with the correct python libraries installed and making sure all libraries needed were installed. This was important because the application maker I used (Pyinstaller) makes the application/executable regardless of if you have the correct libraries installed, so it can possibly make an app that starts up and immediately says no library found. But once I had these made, I tested them out in paths without the required files and without python to make sure that they can run just off of a single download.

Of course, I also worked on all presentations, papers, and group meetings.

4.3 Tomer Carmeli

I began this project by reading the paper regarding how keystroke dynamics can be used for user authentication as an alternative to standard passwords. From here, I was able to get a solid understanding of what algorithms and the general approach the group would need to take in order to successfully complete this project.

I also spent a lot of time with Santi planning out the overall architecture of the project and what each step would look like. We originally started off by looking at open source software and hardware for our project. Initially, we planned on building a PCB since we found documentation for it on SoloKeys' GitHub repository, however, I quickly realized that we would not have enough time or resources to create a schematic, have it delivered, and solder all of the parts. As a result, I found an existing SoloKeys USB stick that accomplishes the same tasks as the PCB would have, at a much more reasonable price and eliminated the unnecessary complexity that came with developing the hardware. In addition to this, the USB still maintains the FIDO-2 compatibility we were aiming for with the PCB.

The majority of my work was on setting up the daemon and integrating it with the keystroke algorithm we used. A daemon is an application that is constantly running as a background process on the operating system. Our goal with this was to be able to identify whenever the Solokeys USB is plugged in, and prompt the user with an authentication sequence. I started off by reading about the design of Daemons from numerous sources and how they perform. I then started creating a script that is able to list all devices plugged into USB ports on my Windows laptop. From there, I was able to build on this by then seeing if the correct device is connected based on its hardware ID, which is specific to SoloKeys devices. If this condition was met, then the user would then be prompted with a basic GUI screen for testing. I then worked in collaboration with Santi to make the necessary modifications for MacOS and Linux based devices, since they required a different Python library. Once the device identification and GUI screen appeared, I was able to turn this into a Daemon, by providing documentation on how to make the script run in the background. All of this work was done using Python.

Another addition I made was addressing a concern we had from both of our presentations about dealing with changing typing patterns over time for a user. To counter this change, I added a certificate system that detects when the SoloKeys USB is plugged into a laptop. If it has been more than three months since the certificate was last updated, the user is prompted to re-enroll as a way of accounting for any typing changes. This is then added to their adaptive enrollment profile, ensuring that our system remains up to date. This also works in tandem with the reset user function that Kyle created in case a user undergoes a drastic change that can impact their typing patterns, such as a broken hand.

Finally, I integrated the main gui and algorithm the rest of the group designed along with my Daemon script. In order to do this, I replaced the test gui function calls I created when I built the Daemon and added the libraries that the rest of the group created. Since they made their gui object-oriented, this allowed for seamless integration and a fully-functioning final system.

4.3 Santiago Alzate

I took ownership of the KLAVIS host-side application and the overall system architecture, including the design of the hardware integration that would extended the prototype into a complete authenticator. My work spanned algorithm implementation, application development, evaluation tooling, and firmware integration design for future work.

Primary contributions

- Algorithm and Engine: Implemented the Manhattan Scaled algorithm scoring engine in python, following the algorithm from Killourhy and Maxion (2009). This included the Bloom Filter pre-check for early rejection attempts with unrecognized bigrams, IQR-based outlier filtering during enrollment, and per-feature scaling using mean absolute deviation. Initial work prototyped a One-Class SVM approach, which was discarded against the literature established Manhattan (scaled) as the appropriate baseline
- Keystroke Capture and feature extraction: Built the keystroke event capture pipeline using pynput, with filtering of non-passphrase keys and extraction of the 18-element feature vector per attempt
- Profile Lifecycle: designed and implemented the 2 phase profile lifecycle. During the growth phase the profile uses equal-weight averaging of all collected samples; Whereas during the adaptation phase exponential moving average (alpha 0.05) to track gradual drift without overreacting to individual attempts. Implemented adaptive phases that use mean average deviation vectors to determine the phase of the profile.
- GUI application: Implemented the full tkinter-based application covering user registration, initial enrollment, ongoing authentication, profile management, benchmark mode, and a developer menu for testing edge cases. Implemented multi-user support with per-user JSON-backed profile persistence.
- Source-code reading and firmware analysis: worked through the fido-authenticator Rust codebase to identify the natural injection point for biometric user verification, locating the central pin_prechecks function as the shared UV gate for both make_credential and

get_assertion flows, and identifying the storage pattern in PersistentState that would inform on-token profile layout

5. Experiments and Results

5.1 Experimental Setup and Environment

The KLAVIS evaluation was conducted using the project's benchmark and visualization pipeline, which records authentication outcomes and summarizes them through a keystroke biometrics dashboard. In each benchmark run, the enrolled user generated genuine samples by typing the selected passphrase, while additional samples were collected from impostor attempts to simulate unauthorized use. For every typing attempt, the system extracted timing-based keystroke features and compared them against the stored user template using the Scaled Manhattan Distance classifier.

To evaluate classification performance, KLAVIS computed a score for each attempt and then performed a threshold sweep to determine the tradeoff between accepting legitimate users and rejecting impostors. From this sweep, the system calculates the False Acceptance Rate (FAR), False Reject Rate (FRR), and Equal Error Rate (ERR) for both individual runs and pooled results across multiple runs. In addition to per-run analysis, the system also tracked profile adaptation over time, threshold convergence, and the relationship between profile sample count and authentication accuracy.

The collected results reflect both single-run behaviour and aggregate behaviour across repeated benchmarks. The aggregate plot shows that the evaluation eventually included 11 benchmark runs, with a pooled total of 20 genuine attempts and 30 impostor attempts in the self-pooled summary. This structure allowed the project to evaluate not only whether KLAVIS works in isolated cases, but also whether its performance becomes more stable as more user data is collected.

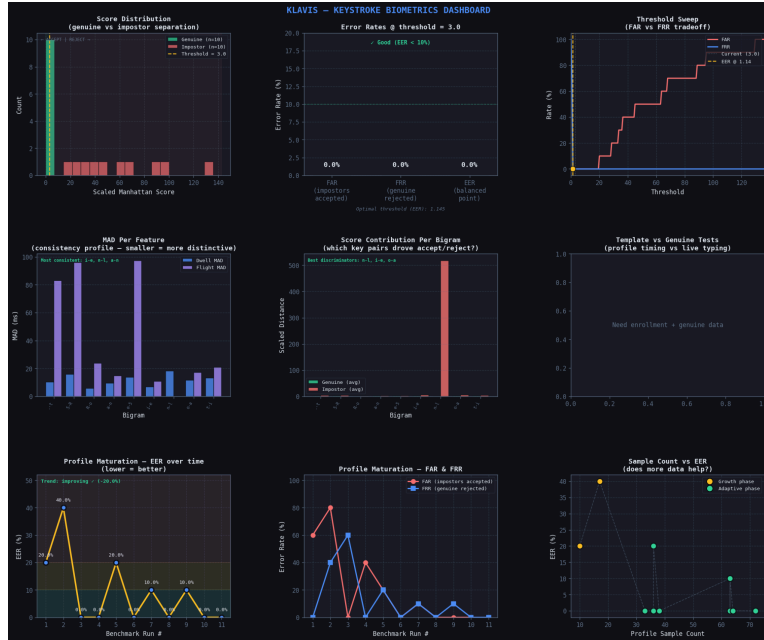


Figure 3: Performance Metrics Dashboard

5.2 Enrollment Phase Results

The enrollment-phase results show that KLAIVIS experiences the expected cold-start problem: when only a small number of user samples are available, the biometric template is still unstable, which leads to higher error rates. In the early runs, the system produced relatively high EER values, including cases around 20% and 40%, indicating that the initial template did not yet capture the user's typing behavior reliably enough.

As more enrollment and post-enrollment samples were added to the profile, performance improved significantly. The Sample Count vs EER plots show a clear downward relationship between the number of profile samples and authentication error. In the aggregate statistics, the fitted trend line indicates an improvement of roughly 0.37–0.38 EER percentage points per additional sample, confirming that more profile data generally leads to better discrimination between genuine and impostor typing. By the time the profile reached roughly the 35–70 sample range, multiple runs achieved 0% EER, which suggests that the user template became substantially more stable after sufficient data had been collected.

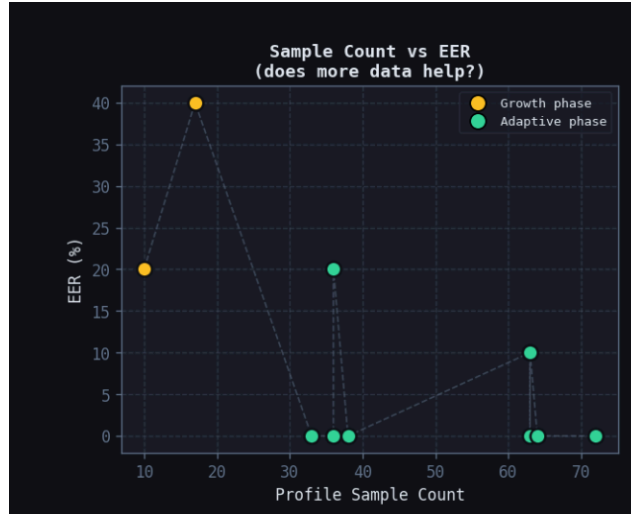


Figure 4: Sample count vs ERR Chart

These results support the motivation for adaptive enrollment and profile growth. Rather than relying on a very small fixed enrollment set, the results suggest that KLAVIS benefits strongly from continued template strengthening during the growth and adaptation phases. In other words, the early enrollment phase is necessary to bootstrap the classifier, but the highest-quality results only emerge once the profile has matured beyond the initial sample set.

5.3 FAR / FRR / EER Metrics

The main authentication metrics used throughout the evaluation were the False Accept Rate (FAR), False Reject Rate (FRR), and Equal Error Rate (EER). FAR measures the percentage of impostor attempts incorrectly accepted, FRR measures the percentage of genuine attempts incorrectly rejected, and EER identifies the threshold at which these two error types are balanced. These metrics provide a compact way to evaluate how well the system distinguishes legitimate users from unauthorized ones.

The aggregate self-pooled results are strong. In one of the all-time pooled summaries, KLAVIS achieved a pooled EER of 0.00% at a threshold of 4.70, using 11 benchmark runs, 20 genuine attempts, and 30 impostor attempts. This indicates that, when the data from repeated self-benchmark runs are pooled together, the system was able to cleanly separate the enrolled user's typing from the impostor samples collected in that setting.

At the single-run level, the results were more variable, especially in earlier runs. Some benchmark dashboards show nonzero FRR or EER values, including 10% and 20% error cases. In these runs, the errors were typically caused by either an immature profile or a threshold that had not yet fully stabilized. However, later runs frequently reached 0% FAR, 0% FRR, and 0% EER, which demonstrates that the system can achieve perfect separation once enough data is available.

5.4 Score Distribution – Genuine Vs. Imposter

The score distribution plots show that the Scaled Manhattan Distance is generally effective at separating genuine and imposter typing attempts. Across most runs, genuine scores remain concentrated near zero, while imposter scores appear much larger, often spread over tens or even hundreds of score units. This behavior is exactly what KLAVIS aims to achieve: legitimate users should produce small distances from their own profile, while impostors should appear far from the stored template.

In the strongest runs, the separation is visually clear, with a large gap between the genuine cluster and the imposter cluster. In these cases, the selected threshold cleanly splits the two populations, resulting in 0% FAR, 0% FRR, and 0% EER. These plots provide some of the strongest visual evidence that the classifier is learning meaningful user-specific timing patterns.



Figure 5: Genuine vs imposter score distribution on a mature profile

However, the score distributions also show why early enrollment and limited data can be problematic. In weaker runs, the impostor scores are still mostly higher than the genuine scores, but the separation margin is smaller, and the threshold has less room for error. This leads to isolated cases in which genuine users are rejected or where the chosen threshold becomes more sensitive to noise. Overall, the score distribution figures strongly support the claim that the classifier works, while also illustrating that reliability improves as the user profile matures.



Figure 6: Impostor vs genuine user score distribution on an immature profile

5.5 Profile Maturation

The profile maturation results show that KLAIVIS improves over time as the stored user template accumulates more evidence. In the maturation plots, the early benchmark runs exhibit higher error rates, including a notable spike to 40% EER in one of the early runs. After that initial unstable period, the results improve substantially, with several later runs reaching 0% EER and only occasional small regressions. In one representative dashboard, the trend is explicitly labeled as improving by approximately 20%, which aligns with the general visual trend of decreasing error over time.

The FAR and FRR maturation plots provide more detail about where these errors come from. In the earlier runs, both false accepts and false rejects appear at meaningful levels, indicating that the profile is not yet well calibrated. As the system collects more data, both metrics tend to decline, and the classifier becomes more stable. Although the improvement is not perfectly monotonic, the overall pattern shows that the profile becomes increasingly reliable after repeated successful interactions.

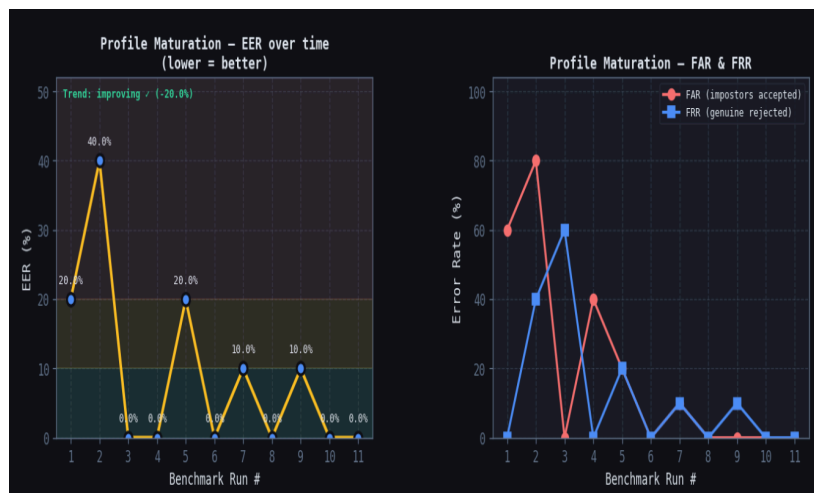


Figure 7: Profile Maturation over time

This behavior is important because it validates the KLAVIS design choice of supporting profile growth and ongoing adaptation. The biometric template should not be treated as static after the first few samples. Instead, the results suggest that continuous refinement is essential for reducing both user friction and security errors. The maturation results, therefore, support the overall system architecture, in which enrollment is only the first phase of a longer profile-stabilization process.

5.6 Best Run Results

The best single-run results demonstrate the full potential of the KLAVIS approach once the biometric profile has matured. In one representative benchmark run, performed at a threshold of 3.0, the system achieved 0% FAR, 0% FRR, and 0% EER. The corresponding score distribution shows the genuine attempts tightly grouped near zero, while impostor attempts appear at significantly larger distances, creating a clean decision boundary.

The threshold sweep for this run also supports the stability of the classifier. Although the dashboard reports an optimal EER threshold of approximately 1.145, the system still achieved perfect classification at the current operating threshold of 3.0 for that run. This suggests that the separation between classes was strong enough to tolerate a range of reasonable threshold values without degrading performance.

The same run also provides useful insight into why the classifier succeeds. The feature-level plots show that some bigrams contribute much more strongly than others to the final score, meaning that specific timing relationships in the user's typing are especially discriminative. In practical terms, this indicates that the classifier is not merely memorizing noise, but is instead capturing stable patterns in the user's typing rhythm. For the report, this best-run result is valuable because it shows that the system is capable of achieving perfect genuine/impostor separation under favorable conditions.



Figure 9: Best Run data chart

6. Video of Demonstration

The following video provides a full end-to-end demonstration of the KLAVIS system in operation. The demonstration covers the complete download, setup, and user workflow including profile creation, the authentication system during both the growth and adaptation phases, benchmark testing with genuine and imposter attempts, and the resulting visualization dashboard showing score distributions. Both GUIs are shown and fully demonstrated, and profiles created by all four group members are used for demonstration purposes. The link below takes you to this video:

Link:  [final_report_demo.mp4](#)

7. Conclusions and Future Work

7.1 Conclusions

KLAVIS demonstrated that keystroke biometric authentication can be meaningfully integrated into a hardware token form factor using lightweight statistical methods that operate effectively at small data scales. The core research question driving the project was whether a system constrained to the practical realities of a real product, where users will not tolerate large enrollment burdens, profiles must adapt to natural typing drift over time, and authentication decisions must be made in real time. The ultimate test was to see if we could meet all of those realities while still achieving error rates approaching those reported in controlled research settings with hundreds of samples per user.

While our testing was not able to come close to the size of testing done by the works that motivated this project, we came to an optimistic answer. We were able to demonstrate that the system is capable of clean separation between genuine and imposter typing patterns when the profile has had enough time to mature. Earlier benchmark runs on immature profiles showed higher error rates, which is consistent with the design intent. The growth phase is specifically shown to be a period of reduced security that improves as the profile grows over time. The adaptive enrollment system successfully differentiated between consistent and inconsistent typists in testing, stopping enrollment early for users with stable typing rhythms while collecting more samples from users with higher variability. The phase transition system correctly identified when profiles had matured based on authentication history rather than arbitrary time or sample thresholds, and the dual threshold design demonstrably prevented the profile poisoning that was observed in earlier single-threshold implementations.

The hardware side of the project proved to be the most challenging constraint. The Solo2 token that we had ordered is the standard version, which does not expose the firmware modification interface needed to integrate the keystroke gate directly into the FIDO2 authentication flow. As a result the software stack runs as a host side daemon rather than on the token itself, which achieves the intended security behavior but does not fulfill the original vision of a fully self contained hardware token where the biometric decision happens on the chip. This remains the main unresolved gap between our current final product and the full KLAVIS described in our previous presentations.

Despite this limitation, the software stack represents a complete and functional implementation of the keystroke biometric layer. The profile system, adaptive enrollment, phase lifecycle, dual threshold authentication, benchmark tooling, visualization dashboard, USB daemon, certificate system, and dual GUI architecture all work together as an integrated system that could be connected to compliant hardware with relatively straightforward firmware work.

7.2 Lessons Learned

The most important lesson from this project was that algorithm selection is directly tied to data scale constraints. Approaches that work well in the literature like SVMs, neural networks, large enrollment sets all fail badly when the practical ceiling on enrollment samples is 8 to 20. Choosing an algorithm that was explicitly benchmarked on small fixed text samples and understanding why it works at that scale was more valuable than chasing higher-performing methods that require data volumes incompatible with the product context.

The second major lesson was that adaptive systems require careful protection against the data they learn from. The profile poisoning problem was not anticipated at the outset and only became apparent through testing. Building the dual threshold defense required recognizing that the learning decision and the access decision are completely different problems with different risk profiles, and that collapsing them into a single threshold creates a subtle but serious vulnerability.

On the engineering side, working across multiple operating systems like macOS for algorithm development and Windows for integration introduced problems when running certain scripts, particularly around pynput accessibility permissions on macOS and WMI availability on Windows. Writing the scripts to run natively on both systems would've saved time by preventing us from going back into working scripts and editing them to work on both. Similarly, keeping track of libraries used would've been useful. Of course you can go into each file individually and write down the library headers and install them, but you can also just make a text file with the libraries used. Then whenever you need to install the libraries, you can just use a recursive terminal command to install the libraries mentioned in the text file.

7.3 Additional Week of Time

With one additional week, the immediate priority would be ordering the Solo2 hacker edition token, which unlike the standard consumer version exposes the firmware interface and allows custom application code to be flashed onto the device. While this version still isn't perfect, as if the firmware breaks at any point, the entire USB is completely unusable, this single item would unblock the entire firmware integration workstream that stalled when the standard token arrived. The week would be spent getting the development environment configured for the chip, verifying that the Rust toolchain and Solo2 firmware build system are working correctly, and establishing a basic data flow from the host daemon to the token. Even just getting a custom string to appear over USB from modified firmware would represent meaningful progress toward the full integration.

7.4 Additional Month of Time

With a full additional month, the focus would shift to actually deploying the keystroke gate logic onto the token firmware and cleaning up the project for a shippable end user experience. The keystroke authentication decision would move from the host daemon into the token itself so that the private key signing operation is only triggered after the biometric check passes on chip rather than on the host machine. On the software side the user GUI would be polished further, the installation process would be simplified into a single makefile or installer script, and the system would be tested across a broader set of users to gather enough benchmark data to produce statistically reliable EER estimates that can be compared directly against the CMU benchmark results.

7.5 Necessary Skills

Rust programming experience would have been the single most impactful skill gap to close. No one in the group had prior experience with Rust, and this would have been very beneficial. The Solo2 firmware is written entirely in Rust and while the language is learnable, the combination of an unfamiliar language, an unfamiliar embedded target, and the complexity of the existing FIDO2 codebase created a steep ramp that consumed a disproportionate amount of time. On the software side, deeper experience with cross platform Python packaging and distribution would have reduced the environment setup friction that came up repeatedly when moving between machines and operating systems. Along with this, further experience with algorithms such as the Manhattan one used in our project would have saved time during the research phase of our project, as we could've eliminated certain methods like machine learning for our scope from the beginning.

References

- [1] Cockell, R., and B. Halak, "On the Design and Analysis of a Biometric Authentication System Using Keystroke Dynamics," arXiv preprint arXiv:1909.10841, 2019
- [2] Rashik Shadman, Ahmed Anu Wahab, Michael Manno, Matthew Lukaszewski, Daqing Hou, and Faraz Hussain. 2025. Keystroke Dynamics: Concepts, Techniques, and Applications. ACM Comput. Surv. 57, 11, Article 283 (November 2025), 35 pages. <https://doi.org/10.1145/3733103>
- [3] López, C., Solano, J., Rivera, E. et al. Adversarial attacks against mouse- and keyboard-based biometric authentication: black-box versus domain-specific techniques. Int. J. Inf. Secur. 22, 1665–1685 (2023). <https://doi.org/10.1007/s10207-023-00711-0>
- [4] SoloKeys, solo2-hw: Hardware Schematic and Layout for Solo 2 [Online]. Available: <https://github.com/solokeys/solo2-hw>. Accessed: Apr. 5, 2026.